

HOW TO IMPLEMENT HASHICORP VAULT FOR SECRET MANAGEMENT IN TERRAFORM

VAULT is a **separate HashiCorp tool used to securely store and manage secrets** (passwords, API keys, database credentials, certificates) instead of hardcoding them in your Terraform files or terraform.tfvars files.

For example, suppose you have an **AWS access key**. A best practice would be **not** to put it directly into your Terraform files because:

- Anyone with access to the code can see it.
- It might get pushed to GitHub accidentally.

Instead, you store it in a **Vault**. Terraform can then retrieve it securely when you need your access keys.

For example, to read a secret from **HashiCorp Vault**, you would use the **vault_generic_secret** data source.

```
data "vault_generic_secret" "aws_access_key_id" { path = "secret/aws/access_key_id" }  
  
variable "aws_access_key_id" { value = data.vault_generic_secret.aws_access_key_id.value }
```

To encrypt the **Terraform state file**, you would first configure a **secure remote backend** for the **state file**. Then, **you would encrypt the state file** using the **terraform encrypt** command.

```
terraform encrypt
```

To use **environment variables**, you would **first define the environment variables** in your **operating system**. Then, you would **configure Terraform** to read the **environment variables** when it is run.

For example, to define an **environment variable** called **AWS_ACCESS_KEY_ID**, you would use the following command:

```
export AWS_ACCESS_KEY_ID=YOUR_ACCESS_KEY_ID
```

Then, you would configure Terraform to read the environment variable by adding the following line to your Terraform configuration file:

```
variable "aws_access_key_id" { source = "env://AWS_ACCESS_KEY_ID" }
```

ENABLE VAULT INTEGRATION BY STORING SENSITIVE SECRETS IN AN S3 BUCKET IN TERRAFORM.

1) LOGIN TO YOUR **AWS ACCOUNT** AND **CREATE A KEYPAIR** CALLED **vault_key_pair**. MAKE SURE THE **PRIVATE KEY** IS **DOWNLOADED LOCALLY** TO YOUR LAPTOP.

2) FOR **WINDOWS** AND **MACOS USERS**, YOUR **vault_key_pair.pem** PRIVATE KEY FILE WILL BE **DOWNLOADED TO YOUR WINDOWS LOCAL MACHINE**.

YOU CAN COPY THIS FILE TO THE **DIRECTORY YOU INTEND TO USE** IN YOUR **LINUX UBUNTU OPERATING SYSTEM** AND **REFERENCE THIS FILE PATH FROM THAT DIRECTORY** IN YOUR **main.tf** FILE.

ALTERNATIVELY, FOR **WINDOWS USERS**, YOU CAN FOLLOW ALONG TO CREATE A PATH FOR YOUR DOWNLOADED **vault_key_pair.pem** PRIVATE KEY FILE IN YOUR **WINDOWS OPERATING SYSTEM** AND **REFERENCE YOUR FILE PATH FROM THERE** BY TYPING THE COMMANDS BELOW.

IN YOUR **LOCAL MACHINE**, SEARCH FOR **CMD**, AND IT OPENS A **POWERSHELL TERMINAL**. TYPE THE COMMAND BELOW.

```
wsl      # this allows you access to your Windows drives (folders/directory) under  
          # /mnt/ in your Linux environment (username).  
          # Alternatively, search for wsl and change to the  
          # Windows drives (folders/directory) by typing the command  
          # cd /mnt/c/Users/HP/Downloads
```

```
ls          # list all the files in your Windows drives (folders/directory)
```

```
cd Downloads  # change to the downloads directory or folder in your windows os
```

```
pwd          # check your current working directory on your Windows machine
```

copy your directory path

ls **# locate your vault_key_pair.pem file**

cd ~ **# exit your Windows drives (folders/directory)**

ls

ls -a **# check if you have any hidden .ssh directories in your**
home directory (edunaking). If you do, go to the next step; if not, create
one by typing mkdir ~/.ssh

cd /mnt/c/Users/HP/Downloads **# switch back to your Windows drive download directory**

mv /mnt/c/Users/HP/Downloads/vault_key_pair.pem ~/.ssh/

move the vault_key_pair.pem file from your Windows drive download directory to the
hidden .ssh directory in your edunaking home directory in WSL.

cd ~ **# exit your Windows drives download directory.**

ls -a **# check for your hidden .ssh directory**

cd .ssh **# change to this hidden .ssh directory**

ls **# you should see your vault_key_pair.pem file**

pwd **# copy the file path in another sheet, you will need it in the future**

 # your complete file path should look like

 # /home/edunaking/.ssh/vault_key_pair.pem

chmod 400 vault_key_pair.pem **# set read permissions for your file**

ls -la vault_key_pair.pem **# check that you now have read permissions**

cd ~ **# exit the .ssh directory**

3) LOG IN TO YOUR AWS ACCOUNT AND CREATE AN EC2 INSTANCE USING THE UBUNTU DISTRO.

GIVE YOUR EC2 INSTANCE ANY NAME. I WILL USE THE NAME vault_instance

UNDER KEY PAIR, CHOOSE vault_key_pair WE CREATED PREVIOUSLY FOR THIS LESSON.

UNDER NETWORK SETTINGS, CREATE A SECURITY GROUP AND ALLOW FOR HTTP, HTTPS, AND SSH. GIVE IT ANY NAME. I WILL USE THE NAME vault_SG

LEAVE THE OTHER DEFAULTS AND LAUNCH YOUR EC2 INSTANCE.

4) LOCATE YOUR HIDDEN .ssh FOLDER OR DIRECTORY IN YOUR LOCAL MACHINE.

TYPE THE COMMANDS BELOW.

ls -a

cd .ssh **# switch to the hidden SSH directory or folder**

ls

```
eval $(ssh-agent)          # start the ssh agent for ssh forwarding
```

```
ssh-add key.pem           # replace key.pem with your own keypair (vault_key_pair.pem)
```

```
ssh-add vault_key_pair.pem
```

```
ssh -A ubuntu@yourEC2.public.ip.address
```

```
ssh -A ubuntu@13.49.238.47  # ssh without storing your private key pair on your  
                             # EC2 Ubuntu instance
```

4) IN YOUR TERMINAL, UPDATE YOUR **UBUNTU SERVER PACKAGE MANAGER** AND INSTALL **GPG**.

TYPE THE COMMAND BELOW.

```
sudo apt update && sudo apt install gpg    # update your Ubuntu server and install GPG
```

The **GPG application** will **encrypt files** so only the intended person can read them, **decrypt files that were encrypted**, **digitally sign files** to prove they came from you, **verify signatures** to confirm a file has not been tampered with, and **verify the software package's authenticity** (most common use case).

5) **DOWNLOAD THE HASHICORP OFFICIAL SIGNATURE KEY**, AND SAVE IT TO YOUR **UBUNTU EC2 INSTANCE** TO ALLOW **HASHICORP TO VERIFY** THAT ANY **TERRAFORM VAULT** SOFTWARE YOU INSTALL LATER IS **GENUINELY FROM HASHICORP** AND HAS NOT BEEN **TAMPERED WITH**.

COPY AND PASTE THE COMMAND BELOW IN YOUR TERMINAL.

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
```

6) VERIFY THAT YOU HAVE THE **HASHICORP OFFICIAL SIGNATURE KEY** INSTALLED. **COPY AND PASTE** THE COMMAND BELOW IN YOUR TERMINAL.

```
gpg --no-default-keyring --keyring /usr/share/keyrings/hashicorp-archive-keyring.gpg --fingerprint
```

7) ADD THE **HASHICORP REPOSITORY** TO YOUR **UBUNTU EC2 INSTANCE** TO ENABLE **VAULT INSTALLATION**. COPY AND PASTE THE COMMAND BELOW IN YOUR TERMINAL.

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]  
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
```

8) UPDATE YOUR **UBUNTU'S SERVER PACKAGE MANAGER** AND INSTALL **VAULT**.

TYPE THE COMMANDS BELOW.

```
sudo apt update
```

```
sudo apt install vault          # install vault on your EC2 instance
```

```
vault                          # this will show all that we can use the vault to achieve
```

9) START THE **VAULT DEVELOPMENT SERVER**. FOR OUR LEARNING PURPOSES, WE WILL USE THE **VAULT DEVELOPMENT SERVER**, BUT IN REALITY, FOR YOUR ORGANIZATION, YOU SHOULD USE THE **VAULT PRODUCTION SERVER**.

TYPE THE COMMAND BELOW TO START THE VAULT DEVELOPMENT SERVER.

```
vault server -dev -dev-listen-address="0.0.0.0:8200"          # start the vault dev server
```

10) OPEN **ANOTHER TERMINAL SESSION** AND **SSH INTO YOUR EC2 INSTANCE** FROM ANOTHER **TERMINAL WINDOW**. TYPE THE COMMANDS BELOW.

```
ls -a
```

```
cd .ssh
```

```
ls
```

```
eval $(ssh-agent) # start the ssh agent for ssh forwarding
```

```
ssh-add vault_key_pair.pem
```

```
ssh -A ubuntu@13.49.238.47 # ssh without storing your private key pair on your  
# EC2 Ubuntu instance
```

YOU SHOULD BE LOGGED IN TO TWO DIFFERENT SESSIONS ON YOUR TERMINAL.

11) CREATE AN ENVIRONMENT VARIABLE FOR YOUR VAULT ADDRESS IN YOUR SECOND TERMINAL.

SWITCH BACK TO THE FIRST TERMINAL SESSION AND COPY

```
export VAULT_ADDR='http://0.0.0.0:8200'
```

AFTER COPYING IT, SWITCH BACK TO THE NEW TERMINAL SESSION AND PASTE IT IN THE NEW TERMINAL SESSION.

12) BY DEFAULT, YOU CANNOT ACCESS THE VAULT ADDRESS USING THE GENERATED PORT 8200 'http://0.0.0.0:8200' FROM YOUR VAULT. TO CHANGE THIS, YOU NEED TO OPEN A CUSTOMIZED INBOUND PORT ON THE SECURITY GROUP ATTACHED TO YOUR EC2 INSTANCE.

13) LOCATE YOUR **UBUNTU EC2 INSTANCE**.

CLICK ON THE **INSTANCE ID URL**.

SCROLL DOWN AND LOCATE THE **SECURITY TAB**, AND CLICK ON YOUR **SECURITY GROUP ID AND NAME**.

CLICK ON THE **INBOUND TAB** AND CLICK ON **EDIT INBOUND RULE**.

CLICK ON **ADD RULE**, UNDER **TYPE**, CHOOSE **CUSTOM TCP**, UNDER **PORT RANGE**, INPUT **8200**, UNDER **SOURCE**, SELECT **ANYWHERE IPV4**, AND CLICK ON **SAVE RULES**.

14) OPEN A **NEW PAGE** IN YOUR **BROWSER** AND **TYPE** THE COMMAND BELOW.

`http://<your-ec2-public-ipv4-address>:8200`

`http://13.49.238.47:8200`

THE **ABOVE** WILL OPEN THE **VAULT USER INTERFACE** IN YOUR **BROWSER**.

15) SWITCH BACK TO THE **FIRST TERMINAL SESSION**, AND YOU SHOULD NOW SEE A ROOT ACCESS TOKEN LIKE:

Root Token: `hvs.LKNNVPOrgVksb1A5dBj1rfiY`

COPY THE **TOKEN ID** LIKE `hvs.LKNNVPOrgVksb1A5dBj1rfiY` AND **PASTE** IT INSIDE UNDER **TOKEN** IN THE **VAULT USER INTERFACE** OPENED IN YOUR BROWSER. MAKE SURE THAT YOUR **METHOD** IS SELECTED AS THE **TOKEN**.

CLICK ON **SIGN IN**, AND YOU SHOULD BE INSIDE YOUR **VAULT USER INTERFACE**.

16) IN YOUR **VAULT USER INTERFACE**, BY YOUR LEFT UNDER **VAULT**, SELECT **SECRETS**.

IN THE **SECRETS ENGINE PAGE**, CLICK ON **ENABLE NEW ENGINE**.

WHAT ARE SECRETS ENGINES:

I) The **Generic (KV) engine** is like a **secure locker** where you **manually store** and **retrieve secrets**. You put the secret in yourself, and **Vault** simply keeps it **safe until you need it**. These allow us to store different kinds of **secrets from different types of platforms**.

For example, if you want to store **SSH keys**, then you will select an **SSH engine**, or if you want to store **key-value pairs** like a **username and password**, then you will enable the **KV engine**.

Here, nothing is **generated automatically**; you stored it yourself. Common things that are stored are **Database passwords, API keys, SSH private keys, Tokens, and Certificates**.

II) The **Cloud Secrets Engine** creates **temporary cloud credentials** whenever someone needs them.

Instead of storing a **permanent cloud password**, **Vault** generates one on demand.

In AWS, for example, a cloud engineer needs to use AWS. Instead of giving them **permanent AWS credentials**, they ask Vault, "I need AWS access."

Vault creates an **Access Key:AKIAXXXXXXXX** and a **Secret Key:abcd123456...**

These credentials might work for only **1 hour**, and after one hour, they stop working automatically. No one has to remember to delete them.

III) The **Infrastructure secrets engines** create **temporary credentials for infrastructure systems** like **databases, SSH access, or messaging systems**.

Instead of **sharing one permanent password** with everyone, **Vault creates unique, short-lived credentials**.

Suppose your company uses a **database**. Instead of giving everyone the **same credentials**:

Username: admin
Password: Admin123

A developer asks Vault, "I need **database access**." Vault creates **temporary credentials**:

Username: user123
Password: xY9@Lm2

These credentials expire after, say, **30 minutes**. When they expire, Vault removes them automatically.

An engineer might also need to log in to a **Linux server**. Instead of using a **single shared SSH key forever**, they ask Vault to generate a **temporary SSH key** that is valid for a **limited time**, making it much safer.

16) **CREATE A KEY-VALUE (KV) ENGINE**. CLICK ON **KV** UNDER **GENERIC**.

IN THE **PLUGIN REGISTRATION TYPE** PAGE, LOCATE **PATH** AND **CREATE A PATH**. THE **PATH** IS WHERE YOU WANT **HASHICORP VAULT** TO **STORE YOUR CREDENTIALS** OR **SENSITIVE VALUES**.

YOU CAN GIVE YOUR **PATH** THE **NAME**, `kvsecret`

LEAVE THE OTHER DEFAULTS AND CLICK ON **ENABLE ENGINE**

17) IN THE `kvsecret` PAGE, CLICK ON **CREATE SECRET**.

UNDER **PATH FOR THIS SECRET**, YOU CAN USE ANY NAME. I WILL USE `sensitive-secrets`

UNDER **SECRET DATA**, LOCATE **KEY**, INPUT `secret_id`, AND INPUT YOUR **OWN NAME** OR **ANY ID** LIKE `12345678910`, IN THE **VALUE** AND CLICK ON **SAVE**.

18) AFTER YOU HAVE **CREATED YOUR SECRET** IN THE **HASHICORP VAULT**, YOU NEED TO **CREATE A ROLE** IN THE **HASHICORP VAULT** SO THAT IT CAN BE **ACCESSED** FROM **TERRAFORM**.

THIS IS **VERY SIMILAR** TO **IAM** IN **AWS**.

CLICK ON **BACK TO MAIN NAVIGATION**, AND UNDER **VAULT**, CLICK ON **ACCESS CONTROL**, AND AUTOMATICALLY, YOU ARE TAKEN TO THE **ACL POLICIES** PAGE. **ACL POLICIES** ARE WRITTEN IN **JSON** BUT USE THE **.tf** **FILE EXTENSION**.

LOCATE **AUTHENTICATION METHODS** UNDER **AUTHENTICATION BY YOUR LEFT**, CLICK ON IT, SCROLL DOWN, AND CLICK ON **ENABLE A NEW METHOD**.

IN THE **ENABLE AN AUTHENTICATION METHOD** PAGE, CLICK ON **APPROLE** AND IN THE **ENABLE AN APP ROLE METHOD** PAGE, CLICK ON **ENABLE METHOD**.

NOTE: YOU CANNOT CONFIGURE YOUR APP ROLE THROUGH THE USER INTERFACE. YOU CAN ONLY DO THAT THROUGH THE COMMAND LINE.

THE APP ROLE IS WHAT ALLOWS YOU TO AUTHENTICATE WITH TERRAFORM.

19) CONFIGURE **TERRAFORM** TO READ THE SECRET YOU CREATED FROM **VAULT**.

CONFIRM THAT YOU HAVE CREATED **AN ENVIRONMENT VARIABLE** FOR YOUR **VAULT ADDRESS** IN THE SECOND TERMINAL.

NAVIGATE TO THE **SECOND TERMINAL SESSION** AND PASTE

```
export VAULT_ADDR='http://0.0.0.0:8200'
```

20) **CREATE A VAULT APP ROLE POLICY** CALLED **terraform**. COPY THE **POLICY BELOW** OR **COPY** FROM THE **20)_terraform_policy.tf** FILE AND **PASTE IT** IN THE SECOND TERMINAL.

```
vault policy write terraform - <<EOF
path "*" {
  capabilities = ["list", "read"]
}

path "secrets/data/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}

path "kvsecret/data/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}

path "secret/data/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}

path "auth/token/create" {
  capabilities = ["create", "read", "update", "list"]
}
```

```
path "auth/approle/login" {
  capabilities = ["create", "read", "update", "list"]
}
EOF
```

THE ABOVE POLICY ALLOWS **TERRAFORM ACCESS** TO THE **SECRETS** WE CREATED IN **VAULT**.

EACH **PATH** SPECIFIES THE **ACCESS TERRAFORM** HAS TO **EACH PATH**.

OBSERVE **MY OWN HASHICORP VAULT SECRET ROLE PATH** IN **TERRAFORM VAULT**.

```
path "kvsecret/data/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}
```

IF SUCCESSFUL, YOU SHOULD GET THE MESSAGE: **Success! Uploaded policy: terraform**

21) CREATE THE **APP ROLE**. TYPE OR **COPY AND PASTE** THE COMMANDS BELOW.

```
vault write auth/approle/role/terraform \
  secret_id_ttl=0 \
  token_num_uses=0 \
  token_ttl=0 \
  token_max_ttl=0 \
  secret_id_num_uses=0 \
  token_policies=terraform
```

THE **ABOVE ROLE** IS CALLED **APP ROLE** AND IS TO BE **USED ONLY** FOR **DEV ENVIRONMENTS** SO THAT YOU DO NOT ENCOUNTER API ERRORS DUE TO TOKENS AND SECRETS LIMITATIONS.

```
secret_id_ttl=0          # secret ID never expires
token_num_uses=0         # a single token can be used unlimited times
token_ttl=0              # token never expires by time
token_max_ttl=0          # no maximum lifetime cap on token
secret_id_num_uses=0     # secret ID can be reused unlimited times
```

IN A **PRODUCTION ENVIRONMENT**, YOU NEED TO SET **THIS** TO FIT YOUR ORGANIZATION'S **PRODUCTION ENVIRONMENT SECURITY POLICIES**.

THE NAME OF THE **APP ROLE** IS **auth/approle/role/terraform** AND THE

```
token_policies=terraform    # name of the "terraform" policy attached
```

SECTION ATTACHES THE POLICY terraform TO THE APP ROLE auth/approle/role/terraform

**IF SUCCESSFUL, YOU SHOULD GET THE MESSAGE: Success! Data written to:
auth/approle/role/terraform**

22) GENERATE **A ROLE ID** (LIKE A USERNAME) AND A **SECRET ID** (PASSWORD) PAIR.

TYPE THE COMMANDS BELOW.

```
vault read auth/approle/role/terraform/role-id    # generate a role id
```

```
vault write -f auth/approle/role/terraform/secret-id    # generate a secret id
```

THE **RESULTS GENERATED FROM THE ABOVE COMMANDS** WOULD ALLOW **TERRAFORM TO AUTHENTICATE WITH VAULT AND COMMUNICATE WITH IT.**

THE **ROLE ID** AND **SECRET ID** ARE **SENSITIVE INFORMATION** AND SHOULD NOT BE SHARED WITH ANYONE. THINK OF THEM LIKE A USER'S **ACCESS** AND **SECRET KEY ID** IN AWS.

23) CREATE A **TERRAFORM RESOURCE.**

OPEN A **THIRD TERMINAL SESSION** AND CREATE A **NEW FOLDER OR DIRECTORY** CALLED **vault**.
TYPE THE COMMANDS BELOW.

```
mkdir vault
```

```
ls
```

```
cd vault
```

nano main.tf

create a configuration file

INSIDE YOUR **main.tf** FILE EDITOR, COPY AND PASTE THE HCL SCRIPTS BELOW. **ALTERNATIVELY**, YOU CAN COPY FROM THE **20)_main.tf** FILE PROVIDED FOR YOU.

```
provider "aws" {
  region = "eu-north-1"
}

# Authenticate terraform with hashicorp vault
provider "vault" {
  address          = "http://13.60.196.181:8200"    # your own EC2 public IPV4
                                                       # address and vault port

  skip_child_token = true

  auth_login {
    path = "auth/approle/login"

    parameters = {
      role_id   = "f5335eb4-e42a-ab1c-55b6-cb6d5e2299de"    # your own role id
      secret_id = "ddc0c520-30b9-9020-191b-502402e9b707"    # your own secret id
    }
  }
}

# create a random id resource suffix for a unique S3 bucket name
resource "random_id" "suffix" {
  byte_length = 4
}

# S3 bucket name is not sensitive and is safe to persist in the terraform.tfstate
# file

resource "aws_s3_bucket" "infra" {
  bucket = "chinedu-secrets-bucket-${random_id.suffix.hex}"

  tags = {
    Name = "s3_vault_infra"
  }
}
```

```

# Block public access to this bucket because it holds sensitive content

resource "aws_s3_bucket_public_access_block" "infra" {
  bucket = aws_s3_bucket.infra.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

# Fetch the secret from Vault so it is never persisted to the terraform.tfstate
file
# ephemeral guarantees this value is never written to state

ephemeral "vault_kv_secret_v2" "s3_secret" {
  mount = "kvsecret"          # name of your path in hashicorp vault
  name  = "sensitive-secrets" # name of your secret in hashicorp vault
}

# Upload the secret to S3 directly using the local exec provisioner.
# this is the supported way to write an ephemeral value to S3 without it touching
the terraform.tfstate file

resource "terraform_data" "upload_secret" {
  depends_on = [aws_s3_bucket_public_access_block.infra] # before terraform
                                                         # uploads the secret
                                                         # the bucket must
                                                         # already be created

# copy the secrets from terraform vault to the created S3 bucket with the secrets
prefix
# or folder and it should be written as an object into a text file called secret-
id.txt

  provisioner "local-exec" {
    command = "echo '${ephemeral.vault_kv_secret_v2.s3_secret.data["secret-id"]}'
| aws s3 cp - s3://${aws_s3_bucket.infra.id}/secrets/secret-id.txt"
  }
}

```

SAVE YOUR FILE.

24) **CREATE YOUR RESOURCES IN AWS. TYPE THE COMMANDS BELOW.**

ls

terraform init

terraform plan

terraform apply

aws s3 ls **# list to see all your created S3 buckets**

aws s3 ls s3://chinedu-secrets-bucket-d57c83c8 **# list to see all the objects in your bucket**

cat terraform.tfstate **# check to see if your secret is hardcoded in the terraform.tfstate file**

terraform destroy **# destroys your secret in Hashicorp Vault and your S3 resource**

ALTERNATIVELY, YOU CAN CHECK THAT YOUR S3 BUCKET WAS CREATED WITH THE MANAGEMENT CONSOLE BY NAVIGATING TO S3 AND DOWNLOADING YOUR SECRET.

FOR MORE INFORMATION, PLEASE VISIT THE HASHICORP DOCUMENTATION ON VAULT USING THE PROVIDED LINKS BELOW.

AppRole Auth Method: <https://developer.hashicorp.com/vault/docs/auth/approle>

KV Secrets Engine v2: <https://developer.hashicorp.com/vault/docs/secrets/kv/kv-v2>

Policies: <https://developer.hashicorp.com/vault/docs/concepts/policies>

